

Doorlink

Development workflow brief — Claude Code ↔ GitHub ↔ Replit

Prepared 16 September 2026 · github.com/mizlaa/doorlink · for the engaged freelancer

This is a workflow and safety job, not a build job. The application is already substantially developed and working. I need the three environments wired together so that design work and backend work can run in parallel without either destroying the other.

1. What I want

A safe, reliable workflow between **Claude Code** ↔ **GitHub** ↔ **Replit**, so I can keep developing the app in Claude Code while using Replit for UI and design, without either one overwriting or breaking the other.

Specifically

- Review the existing codebase for bugs, structural issues, conflicts, or anything that will cause problems later.
- Establish GitHub as the single source of truth.
- Connect the existing project into Replit using **the same repository** — not a separate copy, not a re-import, not a conversion.
- Create a branch workflow so Replit design work and Claude Code development proceed independently.
- Replit is for UI/UX, styling, visual design and interface work.
- Claude Code remains for backend, database, APIs, functionality and future features.
- Changes from either side push and pull through GitHub without overwriting the other.
- All changes go through branches and pull requests. Nothing edits `main` directly.
- Document clearly which files each side owns, and where conflicts are likely.
- Give me a simple step-by-step process for working between the three after handover.

Non-negotiable. A large amount of development is already complete. The existing functionality, source code, database schema and development history must not be lost, replaced, reset or rebuilt. This job is about protecting and connecting what exists.

2. Where the project stands today

Read from the repository on 16 September 2026, so you can quote against real numbers.

Repository	github.com/mizlaa/doorlink
Stack	Next.js 15 (App Router), React 19, TypeScript
Database	PostgreSQL via Prisma 5 — 52 models
Package manager	npm — not pnpm, not workspaces
3D	three.js · @react-three/fiber · drei · postprocessing
Tracked files	224
Routes	59
Library modules	38
Tests	42, all passing (vitest)
Typecheck / build	both pass
Current state of main	clean, current, all PRs merged

Already in the repository. Two documents exist and should be treated as the starting point rather than rewritten from scratch: `WORKFLOW.md` (the three-way workflow) and `FRONTEND_HANDOFF.md` (design-side ownership rules, and a section listing behaviour that must survive any redesign). Review, correct and extend them.

3. Branch tree — the model I want

github.com/mizlaa/doorlink

main

PR #4 (review)

PR #5 (review)

claude/<feature>

backend, db, APIs

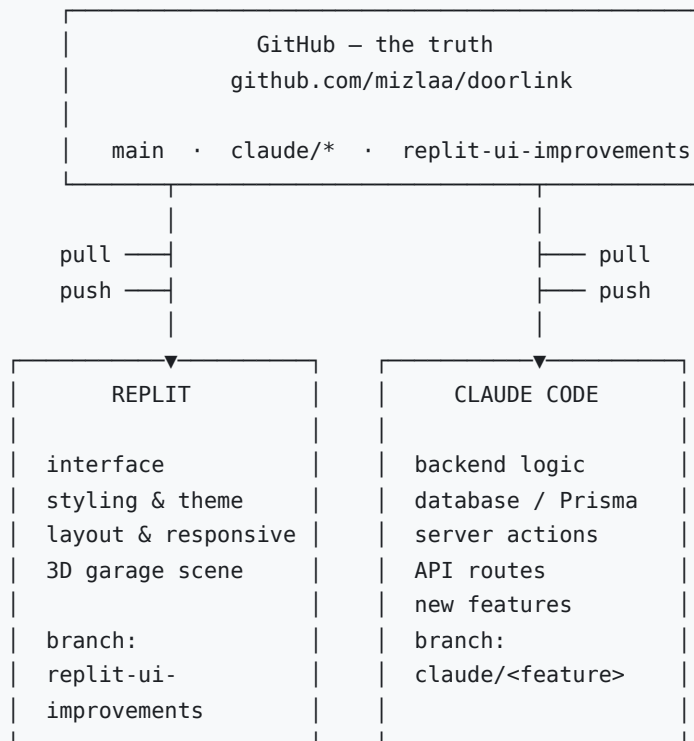
replit-ui-improvements

styling, layout, 3D

Rules

1. Nothing commits to main directly. Ever.
2. main only moves through a merged pull request.
3. Each side branches FROM main and merges BACK to main.
4. Pull from main before starting. Push before stopping.
5. A branch that goes wrong is deleted. main never moved, so nothing is lost.

4. How the three connect



The two environments NEVER talk to each other.
GitHub is the only channel. That is the whole design.

This matters because it is what makes the setup recoverable. Neither tool holds state the other depends on. If a Replit container is lost, nothing is lost — the work is on GitHub. If a design pass is bad, the branch is deleted and `main` is untouched.

5. File ownership map

This is the part I most want documented properly, because it is where the two sides will collide.

src/			
└─ app/	59 routes		
└─ page.tsx		SHARED	markup R · data C
└─ layout.tsx		SHARED	
└─ globals.css		REPLIT	
└─ admin/ leads/ jobs/ ...		SHARED	markup R · actions C
└─ */actions.ts		CLAUDE	server actions
└─ components/			
└─ ui/		REPLIT	design primitives
└─ layout/		SHARED	△ nav arrays
└─ three/		REPLIT	3D garage scene
└─ admin/ auth/ finder/		SHARED	
└─ marketplace/		SHARED	
└─ lib/	38 modules		
└─ marketplace.ts commission.ts		CLAUDE	business rules
└─ rbac.ts auth.ts		CLAUDE	permissions
└─ payments/ configurator/		CLAUDE	
└─ prisma.ts		CLAUDE	
prisma/schema.prisma		CLAUDE	52 models
tailwind.config.ts		REPLIT	theme tokens
package.json		SHARED	△ dependencies
.replit		SHARED	△ see §6

Conflict hotspots — document these explicitly

FILE	WHY IT COLLIDES	RESOLUTION RULE
Header.tsx MobileTabBar.tsx admin/layout.tsx	Each holds a navigation array. Design restructures nav; backend adds entries for new screens. Both edit the same array.	Keep every entry from both sides. They are permission-gated — dropping one hides a working screen from the role that needs it.
package.json	Design adds a font or animation library; backend adds a server dependency.	Union of both. Never take one side wholesale.
Route page.tsx files	Design changes the markup; backend changes the query feeding it.	Design owns everything inside return() . Backend owns everything above it.
.replit	Replit rewrites this file on import. See §6.	Must survive as the npm/Next version.

6. Known problem — read before importing

Replit's import has broken this project three times. This is the single biggest risk in the job and the main reason I am hiring for it.

Replit detects Next.js, infers a Vercel project, and offers to "port" it into its own `PNPM_WORKSPACE` stack. Accepting is not an import — it is a framework migration. It:

- moves the entire application into `.migration-backup/`
- replaces the root with an empty Express + Drizzle + pnpm scaffold
- overwrites `.replit` with `[agent] stack = "PNPM_WORKSPACE"`
- writes a `replit.md` describing a stack this project does not use

It is recoverable — the application survives intact inside `.migration-backup/`, and moving it back to the root plus deleting the scaffold restores everything. It has been recovered once already. But it must be prevented, not repeatedly undone.

What I want from you here. A durable fix, not "remember to click Cancel". Work out why Replit's importer applies the scaffold and configure the repository so it imports correctly and stays correct. If that is genuinely not possible, tell me so plainly and document the manual recovery instead — I would rather have an honest limitation than a fix that quietly stops working.

Also note. Three Replit workspaces for this project disappeared from my account on 16 September. Cause unknown — possibly a credit or plan limit. Worth establishing before building a workflow that depends on a Replit container persisting.

7. Step-by-step process I want at handover

Written for me, not for a developer. Assume I am on a phone.

Going to Replit to design

1. Open the Repl. Pull from `main`.
2. Confirm the branch is `replit-ui-improvements`, not `main`.
3. Design.
4. Commit and push before stopping — even mid-way.

Coming back to Claude Code

1. Open a pull request from the design branch.
2. Review, resolve any conflicts per \$5, merge to `main`.
3. Claude Code pulls `main` and continues on a fresh `claude/*` branch.

Setting up a fresh Repl, if one is ever needed

```
npm install
npm run db:push      # create tables from prisma/schema.prisma
npm run db:seed      # load demo catalogue and accounts
npm run dev
```

A 500 on the home page with a missing-table error means db:push has not run. That failure is deliberate and should stay loud — a reachable but empty database is a broken deployment, not an outage.

8. Scope — what this job is and is not

IN SCOPE	NOT IN SCOPE (QUOTE SEPARATELY)
• Codebase review — bugs, structure, conflict risks • GitHub as source of truth • Correct Replit connection to the same repo • Branch and pull-request workflow • Branch protection on <code>main</code> • File ownership documentation • Fixing or documenting the import problem (\$6) • Plain-language working guide for me	• Building the UI or redesigning screens • Implementing Supabase • Replacing the dev-only authentication • Connecting a payment provider • Rebuilding the 3D scene • New features of any kind

9. Context you should know before quoting

Deliberately switched off — do not "fix" these

Several things look unfinished but are intentional, and the reasoning is documented in the code. Please do not treat them as bugs:

Payments	Provider-agnostic. The Stripe adapter throws <code>PaymentsNotConfiguredError</code> , with each method commented with its exact replacement call and a connection runbook.
Subscriptions	No Subscribe button, because there is nothing to charge with.
Entitlements	Feature gate built, returns an empty set by default.
"Not connected" states	Appear across roughly 51 files. These are honest states, not missing work.

The project follows a rule throughout: **nothing may appear functional when the functionality behind it does not exist**. Please preserve that. If a change would make a dead button look live, it is wrong even if it looks better.

Genuine gaps — known, not part of this job

- **Authentication is dev-only** — a plain cookie session with no password, which refuses to run in production. This is the real blocker before launch and is the Supabase conversation, later.
- **The 3D garage is a placeholder** — blocky, with cartoon foliage. It works but is not the intended design. That is the design pass, later.
- Worker replies to reviews, message attachments, and the user-facing report flow are not built.

10. What I need back from you

1. Review the project first.
2. A **fixed quote** and a **timeframe**.
3. Tell me if anything above is impractical, or if you would do it differently. I would rather hear that now than be told later.

The end result I want. I can open Claude Code and continue development, or open Replit and work on the interface, and GitHub keeps everything organised, synchronised and safe.